

Bit Manipulation

NOT($\sim$):- $N = 5 = (101)_2$
 $\sim N = \sim 5 = \sim(101)_2 = (010)_2 = 2$

AND($\&$):- works on two equal-length bit pattern
 $A = 5 = (101)_2$, $B = 3 = (011)_2$
 $A \& B = (001)_2 = 1$

OR($|$):- works on two equal-length bit pattern
 $A = 5 = (101)_2$, $B = 3 = (011)_2$
 $A | B = (111)_2 = 7$

XOR($\wedge$):- works on two equal-length bit pattern
 $A = 5 = (101)_2$, $B = 3 = (011)_2$
 $A \wedge B = (110)_2 = 6$

Left shift($\ll$): shift bit toward left & append 0 at end. It is equivalent to multiplying with 2^k

$$\begin{aligned} 1 \ll 1 &= 2 = 2^1 \\ 1 \ll 2 &= 4 = 2^2, 1 \ll 3 = 8 = 2^3 \\ 1 \ll 4 &= 16 = 2^4 \\ &\vdots \\ 1 \ll n &= 2^n \end{aligned}$$

*In all shifting is toward right side

Right shift($\gg$): shift bit toward right & append 1 at the end. equivalent to dividing with 2^k .

$$\begin{aligned} 4 \gg 1 &= 2 \\ 6 \gg 1 &= 3 \\ 5 \gg 1 &= 2 \\ 16 \gg 4 &= 1 \end{aligned}$$

1. check given no. is power of 2 or not.
 → M1 → repeatedly divide N by 2 if N is even IF we end up with a 1 then N is power of 2 otherwise not [Handle case of 0 also]

bool isPowerOfTwo(int x)

```

{
  if (x == 0)
    return false;
  else
    while (x % 2 == 0)
      x = x / 2;
  return (x == 1);
}

```

★ **Q1**

M2 →

$$x = 4 = (100)_2$$

$$x-1 = 3 = (011)_2$$

$$x \& (x-1) = 4 \& 3 = (100)_2 \& (011)_2 = (000)_2$$

$$\text{Let, } x = 6 = (110)_2$$

$$x-1 = 5 = (101)_2$$

$$x \& (x-1) = 6 \& 5 = (110)_2 \& (101)_2 = (100)_2$$

bool isPowerOfTwo(int x)

```

{
  return (x & !(x & (x-1)));
}

```

*(x-1) can be obtained by simply flipping all the bits to the right of rightmost 1 and also including rightmost 1

★ **Q2**

2. Count the number of ones in binary representation

M1 → Convert to binary & count number of 1

$O(\log n)$

M2 →

$$n = 23 = \underline{10111}_2$$

count = 0

$$n \& (n-1) = \underline{10111} \& \underline{10110} = \underline{10110} \xrightarrow{22} 1$$

$$\downarrow \quad \downarrow$$

$$23 \quad 22$$

$$22 \& 21 = 10110 \& 10101 = \underline{10100} \xrightarrow{20} 2$$

$$20 \& 19 = \underline{10100} \& \underline{10011} = \underline{10000} \xrightarrow{16} 4$$

* no. of bits are reducing in every operation


```
int count_one(int n)
```

```
{ while(n)
```

```
{ n = n & (n-1);
```

```
count++;
```

```
}  
return count;
```

Q7

Q3. Check if i th bit is set in the binary form or not.

→ * in 2^i only i th bit is 1

Perform $\&$ and know about i th-bit.

```
IF (N & (1 << i))  
return true;
```

This Trick
can be used
in Javacaps
Q7

4. generate all possible subsets of a set

→ $A = \{a, b, c\}$

0 = $(000)_2 = \{\}$

1 = $(001)_2 = \{c\}$

2 = $(010)_2 = \{b\}$

3 = $(011)_2 = \{b, c\}$

4 = $(100)_2 = \{a\}$

5 = $(101)_2 = \{a, c\}$

6 = $(110)_2 = \{a, b\}$

7 = $(111)_2 = \{a, b, c\}$

void possibleSubsets(char A[], int N)

```
{ For (int i = 0; i < (1 << N); ++i)
```

```
{ For (int j = 0; j < N; ++j)
```

```
IF (i & (1 << j))
```

```
cout << A[j] << " ";
```

```
}  
cout << endl;
```

5. Find the largest Power of 2 (most significant bit in binary form), which is less than or equal to the given number N.

Bin
(2)
+
index
Pos

return (count)

3. Check if the bit is set in the binary form

if the bit is 1

return x and know about the bit

if (n & (1 << i))

return true;

4. Generate all possible subsets of a set

→ A = {a, b, c}

0 = (000) = 0

1 = (001) = 1

2 = (010) = 2

3 = (011) = 3

4 = (100) = 4

5 = (101) = 5

6 = (110) = 6

7 = (111) = 7

5. Find possible subsets (of A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z)

for (int i = 0; i < (1 << n); ++i)

for (int j = 0; j < (1 << n); ++j)

if (i & (1 << j))

cout << A[j] << " ";

cout << endl;

Binary numbers

$$(274)_{10} \rightarrow 2 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

$$200 + 70 + 4 \rightarrow 274$$

index $\leftarrow$ Posⁿ

$$(101)_2 \rightarrow 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$\rightarrow 4 + 0 + 1$$

$$\rightarrow 5$$

2	14	0
2	7	1
2	3	1
	1	

$$(1110)_2$$

$$\begin{matrix} 18 \\ 6 \\ 3 \\ 1 \end{matrix}$$

$$(1101)_2$$

1 For odd
0 For even

Addition / Subtractions

$$\begin{array}{r} 2 \quad 1 \quad 1 \\ 2 \quad 7 \quad 4 \\ + 9 \quad 8 \quad 7 \\ \hline 12 \quad 6 \quad 1 \end{array}$$

$$\begin{array}{r} 1 \quad 1 \quad 1 \\ 1 \quad 0 \quad 1 \\ + 1 \quad 1 \quad 1 \\ \hline 1 \quad 0 \quad 0 \end{array}$$

negative inverse

$$\begin{array}{r} 9 \\ - 7 \rightarrow 9 + (-7) \\ \hline 2 \end{array}$$

$\rightarrow 2$

$$\begin{array}{r} 1100 \rightarrow 12 + (-5) \\ - 101 \\ \hline \end{array}$$

$$\begin{array}{r} 0001100(12) \\ - 11101(-5) \\ \hline 001111 \end{array}$$

Carry 0
fill 32 bit
after 32 bit I will
be discarded

2's Complement

$$\begin{array}{r} 101 \\ \rightarrow 11111010 \\ + 1 \\ \hline 11111011 \end{array}$$

(way of finding inverse modulo)
1. 2's complement
2. 1's complement
3. Add one

equivalent of -5

9
-3
6

1001 (-11) → --00011
+ 1101 (-11100)

11101 (-11101)

1001
+ 1101

-000110 ⇒ 6

⊗ (and), 1(0), 1(xor)

a	b	&	^
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

(or)

a	b	
0	0	0
0	1	1
1	0	1
1	1	1

~5 →
~(101) → 11010
4P to 32 bit

12 >> 2
1100
0011 → 2 times shift
L → ③

12 << 2
1100
110000 → ④

Right shift $\Rightarrow$ Nos get divided by 2

```
int a = 5;
int b = a >> 1
```

5 $\rightarrow$ 1 0 1

2 $\rightarrow$ 0 1 0

1 $\rightarrow$ 0 0 1

0 $\rightarrow$ 0 0 0

```
int a = 3;
```

```
int b = a << 1;
```

3 $\rightarrow$ 1 1

6 $\rightarrow$ 1 1 0

12 $\rightarrow$ 1 1 0 0

bit operators are Fast

odd/even number

$\curvearrowright$ modulo is little expensive

```
IF (a % 2 == 0) {
```

Number is even

{

else {

Number is odd

}

Do masking

1 1 1

& 0 0 1

0 0 1

1 1 0

& 0 0 1

0 0 0

Just check last bit

2 $\rightarrow$ 1 0

4 $\rightarrow$ 1 0

6 $\rightarrow$ 1 1 0

3 $\rightarrow$ 1 1 1

7 $\rightarrow$ 1 1 1

9 $\rightarrow$ 1 0 0 1

For odd last bit is 1

```
IF (a & 1 == 0) {
```

Number is even

{

else {

Number is odd.

}

Swap Two Number

```
int a = 5;
```

```
int b = 7;
```

```
int temp = a;
```

```
a = b;
```

```
b = temp;
```

```

101
^ 111
a → 010 = 2

```

```
a = a ^ b = 2
```

```
b = a ^ b = 5
```

```
a = a ^ b = 7
```

```

010
111
101 = 5

```

```

010
101
111 = 7

```

without using
third variable

Set ith bit

→

Bit Masking

1. Bit Masking

2. Find ith bit

3. set ith bit

4. clear ith bit

5. Find number of bits to change to convert a to b.

Find ith bit

```

8 7 6 5 4 3 2 1 0
n → 1 0 0 1 1 0 1 0 1

```

M1 → Right shift itime, do & with 1 IF even
then bit is 0 & IF odd then it is 1

M2 → left shift 1 i times

```

8 7 6 5 4 3 2 1 0
n → 1 0 0 1 1 0 1 0 1

```

```
mask → 0 0 0 1 0 0 0 0 0
```

n & mask → Non zero → 1
else → 0



010

010

111

101

101 = 5

1117

without using
third variable

$$\text{mask} = 1 \ll i$$

Set ith bit

8 7 6 5 4 3 2 1 0
n → 1 0 0 1 1 0 1 0 1

set bit

8 7 6 5 4 3 2 1 0
n → 1 0 0 1 1 0 1 0 1

mask → 000001000

100111101

$$\text{mask} = 1 \ll i$$

$$n = n \& \text{mask}$$

Clear ith bit

8 7 6 5 4 3 2 1 0
n → 1 0 0 1 1 0 1 0 1
mask → 1 1 1 1 0 1 1 1 1
100100101

clear a to b.

$$1 \ll i \rightarrow 0000010000$$

$$\sim(1 \ll i) \rightarrow 1111101111$$

IF even
then it is 10

$$\text{mask} = \sim(1 \ll i)$$

$$\text{int res} = n \& \text{mask}$$

Number of bits to convert a to b.

a → 1 0 1 1 0

b → 1 1 0 1 1

1 1 1 1 → 3 (log)

diff should give
strike of xor

a & b → 0 1 1 0 1 → Now count set bits?

in a = 13 → 1 1 0 1 Count 1 ??

M1 →

1 1 0 1

→ (111)

Right shift
every time

0 1 1 0

↓
0 0 1 1

↓
0 0 0 0 (stop until 0)

TC → (log₂) + 1

similar to no.
of digits in
as no getting
divided

Time complexity
reduced from previous

M2 →

n & (n-1) ⇒ it change LSB set bit
to 0

Trick

0 0 0 0 1 1 0 1

↙ 1 1 0 0

↙ 1 0 0 0

↙ 0 0 0 0

TC → (No. of set bit) ←

ex: 13

1 1 0 1

& 1 1 0 0 (12)

1 1 0 0

& 1 0 1 1

1 0 0 0

& 0 1 1 1

0 0 0 0

* In Bit manipulation, IF you know that Particular technique then Qⁿ strikes [have more tricks]

Property:-

(i) $5 \wedge 5$

(ii) $0 \wedge n = n$

$$\begin{array}{r} 101 \\ \wedge 101 \\ \hline 000 \rightarrow 0 \end{array}$$

total pairs OF XOR

set bits?

similar to no. of digits in a as no getting divided

1. Find the only non-repeating element in an array where every element repeats twice.

$a = [5, 4, 1, 4, 3, 5, 1]$

int res = 0;

$$\begin{aligned} \text{res} &= 5 \wedge 1 \wedge 1 \wedge 4 \wedge 4 \wedge 3 \wedge 5 \wedge 1 \\ &= 5 \wedge 1 \wedge 3 \wedge 1 \wedge 3 \\ &= 1 \wedge 3 \wedge 1 \\ &= 3 \end{aligned}$$

QT

2. Find the two non-repeating element in array where every element repeats twice.

$a = [5, 4, 1, 4, 3, 5, 1, 2]$

right = $\text{res} \Delta \sim (\text{res} - 1)$

5 bit set for not

res = 13 ^ 2

$$\begin{array}{r} 11 \\ \wedge 10 \\ \hline 01 \end{array}$$

both bit are different

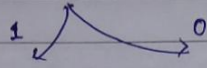
↑ 0 index
Find right most set bit

Edging long for array

divide array in two parts whose 0th index bit is 1 or 0

$$\begin{array}{l} 1 \quad 0 \\ 3 \wedge 12 \wedge 5, 1, 3, 5, 1 \quad 3 \wedge 2 \wedge 4, 4, 2 \end{array}$$

temp = res = 312



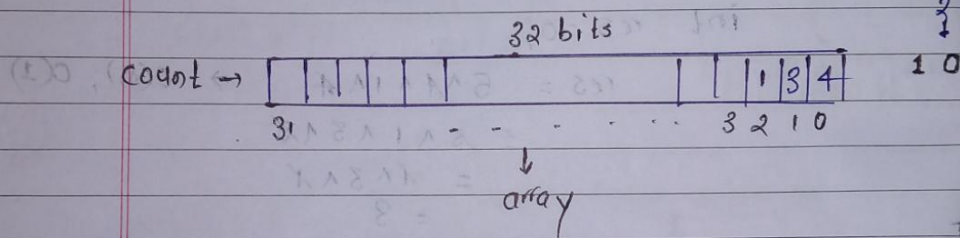
312 1 5, 1, 3, 5, 1 1, 1, 2 1 (312)

(2)

(3)

③ Find the only non-repeating element in array where every other element repeat thrice. $O(N), O(N)$

→ $a = [2, 2, 1, 5, 1, 1, 2]$ $O(N), O(1)$



10	10
10	10
10	10
<u>30</u>	<u>30+1</u>

④ 'Q' can be repeating K times

easy Practice on GFG

8. Find

3	0 → 0
2	1 → 0
	2 → 0
	3 → 0
	4 → 0
	5 → 0
	6 → 0
	7 → 0
	8 → 1
	9 → 1
	10 → 1
	11 → 1

Q. Find Total set bit 1 to N.

Naive approach +

o log n ← Count set bit of every number from 1 to N.

3	0 → 0000
2	1 → 0001
	2 → 0010
	3 → 0011
	4 → 0100
	5 → 0101
	6 → 0110
	7 → 0111

$$4 + 4 + 4 = 2^{x-1}$$

$$2^{x-1} + (n - 2^{x-1} + 1) + solve(2)$$

$$s(n) = 2^{x-1} + (n - 2^{x-1} + 1) + solve(n - 2^{x-1})$$

8 → 1000	→ 000 → 0
9 → 1001	→ 001 → 1
10 → 1010	→ 010 → 2
11 → 1011	→ 011 → 3

5 bit
set
out

int Largest Power of 2 in range (int n) {
int x = 0;

while ((1 < x) <= n)

x += 1;

}

return x - 1;

10

$$2^0 = 1 \leq 10$$

$$2^1 = 2 \leq 10$$

$$2^2 = 4 \leq 10$$

;

```

int solution(int n)
{
    if(n == 0) return 0;

    int x = largestPowerOf2InRange(n);
    int bTill2x = x * (1 << (x-1));
    int msb2xToN = n - (1 << x) + 1;
    int rest = n - (1 << x);

    int ans = bTill2x + msb2xToN + solution(n)
    return ans;
}

```

②

$f^n(\text{arr}[i], \dots)$

{

int xor = arr[0]

int set-bit = 0;

int i; int x=0, y=0;

for (int i=1; i<n; i++)

xor = arr[i]

set-bit = 0 = xor & ~(xor-1);


```

for (int i = 0; i < n; i++)
{
    if (arr[i] & set-bit-no)
    {
        x = x ^ arr[i];
    }
    else
    {
        y = y ^ arr[i];
    }
}

```

olution(x)

③

th bit
s set
not

()

Ques. Find the sum of the first 10 terms of the A.P. 2, 4, 6, 8, ...

Sol. Given: $a = 2$, $d = 2$, $n = 10$

Sum of first n terms of A.P. is given by

1. $S_n = \frac{n}{2} [2a + (n-1)d]$

2. $S_n = \frac{n}{2} (a + a_n)$

- 1) Factor
 - 2) Term
 - 3) Product
- 12

30

No

Palindrom

Mathematics & Number Theory

1) Factorial of a number

2) Trailing no. of zeros

3) Palindromic number

Notice $\rightarrow$ 5 comp
at interval
of 5

$$12! = 1 \times 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8 \times 9 \times 10 \times 11 \times 12$$

$$= 479001600$$

$$\left\lfloor \frac{12}{5} \right\rfloor = 2$$

$$30! = 1 \times 2 \times 3 \times 4 \times 5 \times \dots \times 25 \times 26 \dots 30$$

$$\text{No. of trailing zeros} = \left\lfloor \frac{n}{5} \right\rfloor + \left\lfloor \frac{n}{25} \right\rfloor + \left\lfloor \frac{n}{125} \right\rfloor$$

$$+ \left\lfloor \frac{n}{625} \right\rfloor + \dots$$

For (int i = 5; i <= n; i = i * 5)

$$\text{res} = \text{res} + n/i;$$

return res;

Palindrome number

363

7887

$\rightarrow$ reverse & check (if no.s are equal)

th bit
is set or
not

());

- 1) Prime Numbers
- 2) Sieve of Eratosthenes
- 3) Euclid GCD
- 4) MODULO Arithmetic
- 5) Fast Power
- 6) Comput $a^b \text{ modulo } 10^9+7$

N Prime



No Factor

$1 \rightarrow \sqrt{N}$

$O(N) \rightarrow O(N^{1/2})$

↓ optimize

1 to 12 → Count No. of Prime

1 2 3 4 5 6 7 8 9 10 11 12

boolean[]: F T T T T T T T T T T T

↓ means
Sieve of

eratosthenes

↓
Name

1 2 3 4 5 6 7 8 9 10 11 12
T T T F T F T F T F F F



Now for 3 & so on



isPrime[i] = false; Run this loop (but ~~not~~)

For (int i = 2; i <= n; i++) {

For (int j = 2*i; j <= n; j += i)

{ isPrime[j] = false;

}

}

GCD

Mod

GCD

$$\text{Gcd}(a, b) = \text{gcd}(a-b, b) \quad a > b$$

$$\text{GCD}(a, b) = \text{GCD}(b, a \% b)$$

$$a \% b \neq 0$$

int gcd(int a, int b) {

if (b == 0) return a;

return gcd(b, a % b);

}

loop

Modulo Arithmetic

$$\begin{array}{r} 4 \overline{) 7} \\ 4 \\ \hline 3 \end{array}$$

$$a^b = (a^2)^{b/2} \quad \text{b is even}$$

$$a^b = a \cdot a^{b-1} \quad \text{b is odd}$$

$$5^5 = 5 \cdot 5^4$$

$$5^4 = (5^2)^2$$

th bit
is set or
not

0)

```

int fastPower (int a, int b) {
    int res = 1;
    while (b > 0) {
        if ((b & 1) != 0) {
            res = res * a;
        }
        a = a * a;
        b = b / 2;
    }
    return res;
}

```

modulo

Property :-

- $(a+b) \% m = (a \% m + b \% m) \% m$
- $(a * b) \% m = (a \% m * b \% m) \% m$

same with -ve

$10^{12}, 10^9, 10^9$ it can overflow too

$\Rightarrow (a \% m * b \% m) \% m$
 after performing modulo sum
 $\frac{10^9}{10^9} * \frac{10^9}{10^9}$
 overflow

solution $\Rightarrow$ initially take $a \times b$ as
 long long



long FastPower (long a, long b, int d) {

long res = 1;

while (b > 0) {

if ((b & 1) != 0) {

res = (res * a) % d;

}

a = a * a; → a = (a * a) % d;

b = b >> 1;

}

return res;

}

To avoid negative no

in C++

$(a \% b + b) \% b$

→

To learn apply (do Qⁿ) in binary-search
implement powerⁿ

th bit
is set or
not

(1) :



Date _____
Page _____

For the first part of the problem

$$1 = 2x - 1$$

$$2(0 < d) < 1$$

$$2(0 < d) < 1$$

$$2(0 < d) < 1$$

$$2(0 < d) < 1$$

$$2(0 < d) < 1 \leftarrow 2(0 < d) < 1$$

1.1

$$2(0 < d) < 1$$

$$2(0 < d) < 1$$

$$2(0 < d) < 1$$

to find the value of d

$$2(0 < d) < 1$$

$$2(0 < d) < 1$$

to find the value of d


```

struct Node {
    Node *links[26];
    bool flag = false;
    bool containsKey(char ch) {
        return (links[ch - 'a'] != NULL);
    }
    void Put(char ch, Node *node) {
        links[ch - 'a'] = node;
    }
}

```

```

class Trie {
private:
    Node *root;
public:
    Trie() {
        root = new Node();
    }
    void insert(string word) {
        Node *node = root;
        // TC → O(len)
        for (int i = 0; i < word.length(); i++) {
            if (!node->containsKey(word[i])) {
                node->Put(word[i], new Node());
            }
            // node to reference here
            node = node->get(word[i]);
        }
        node->setEnd();
    }
    bool isEnd() {
        return flag;
    }
    Node *get(char ch) {
        return links[ch - 'a'];
    }
    void setEnd() {
        flag = true;
    }
}

```




Date _____
Page _____

// TC $\rightarrow O(140)$

bool search (string word)

Node * node = root;

For (int i = 0; i < word.length(); i++) {

if (!node -> containKey (word[i])) {
return false;

node = node -> get (word[i]);

return node -> isEnd();

// TC $\rightarrow O(140)$

bool startWith (string PreFix)

Node * node = root;

For (int i = 0; i < word.length(); i++)

if (!node -> containKey (word[i])) {
return false;

node = node -> get (word[i]);

return true;

}

Q. Implement Trie II

Keywords → Count - words - equal - to,
count - words - starting - with
erase - word =

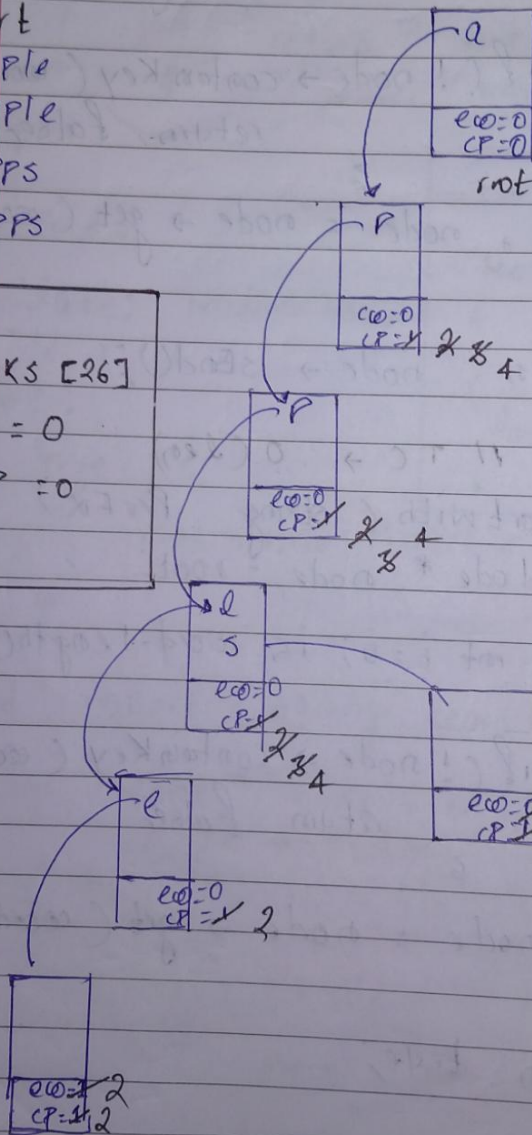
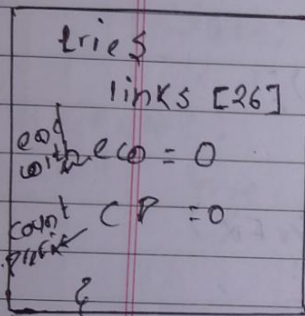
Insert

apple

apple

apps

apps



- with

```

struct Node {
    Node* links[26];
    int cntEndWith = 0;
    int cntPrefix = 0;

    bool containKey(char ch) {
        return (links[ch - 'a'] != NULL);
    }

    Node* get(char ch) {
        return links[ch - 'a'];
    }

    void put(char ch, Node* node) {
        links[ch - 'a'] = node;
    }

    void increaseEnd() {
        cntEndWith++;
    }

    void increasePrefix() {
        cntPrefix++;
    }

    void deleteEnd() {
        cntEndWith--;
    }

    void reducePrefix() {
        cntPrefix--;
    }
}

```

```

class Trie {
private:
    Node* root;
public:
    Trie() {
        root = new Node();
    }
}

```

```
void insert ( string & word)
{
    Node * node = root;
```

```
    For ( int i = 0; i < word.length(); i++)
    {
```

```
        if ( ! node -> containKey ( word[i] ) )
```

```
        {
            node -> Put ( word[i], new Node );
```

```
        }
```

```
        node = node -> get ( word[i] );
```

```
        node -> increasePrefix();
```

```
    }
```

```
    node -> IncreaseEnd();
```

```
}
```

```
int countWordsEqualTo ( string & word )
```

```
{
```

```
    Node * node = root;
```

```
    For ( int i = 0; i < word.size(); i++)
```

```
    {
        if ( ! node -> containKey ( word[i] ) )
```

```
        {
            node = node -> get ( word[i] );
```

```
        }
```

```
        else
```

```
        {
            return 0;
```

```
        }
```

```
    }
```

```
    return node -> getEnd();
```

```
}
```


Date / /
Page

int CountWordsStartingWith(string & word) {
Node * node = root;

For (int i = 0 ; i < word.size() ; i++) {

if (node->ContainKey(word[i])) {
node->get(word[i]);

else return 0;

return node->getPrefix();

void erase(string & word)
{
Node * node = root;

For (int i = 0 ; i < word.size() ; i++) {

if (node->ContainKey(word[i]))

{
node = node->get(word[i]);

node->reducePrefix();

else { return; }

node->deleteEnd();



Date _____
Page _____

* Q. Count Distinct substrings including empty

ex: ^{Substring starting with}
✓ a ✓ b

✓ ab

ba

a x

b x

abab

a ba

bab

a b x

abab

↓
duplicates

7+1 = 8 → ans

↓
For NULL

Brute Force

↓

a b a b

set <string> st

↑ ↑ ↑ ↑
i i i i

For (i = 0 → n-1) → O(N)

2 str = ""

for (j = i → n-1) → O(N)

3 str = str + s[j]

6 st.add(str) → log(N)

6

Print (st.size()) → distinct

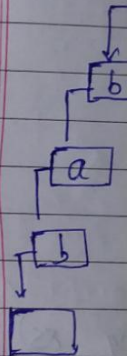
TC → $N^2 \times \log M$

M = No. of
for set → string
in set

SC →

min substring length → 1
max → n

On an
avg $\approx (n/2)$





Date / /
Page

Assuming n^2 substring (near)

$$TC \rightarrow \frac{n^2 \times n}{2}$$



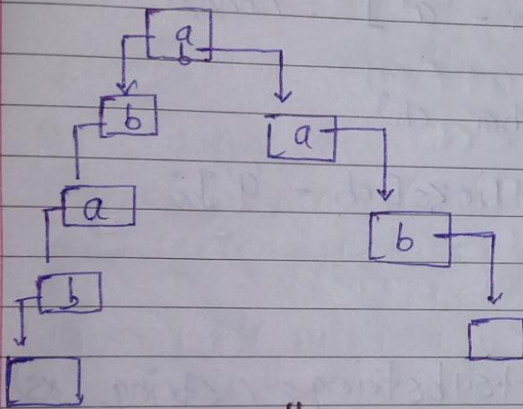
By storing in Trie

a b a b

Trie

arr[26];

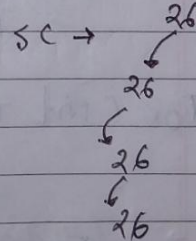
6



$$T+1 = 8$$

$$TC \rightarrow O(N^2)$$

as ³ insert one word ~~each~~ bit



→ ith bit is set or not

Node()



```
struct Node {
```

```
    Node *links[26];
```

```
    bool containsKey(char ch) {
```

```
        return links[ch - 'a'];
```

```
    }  
    void put(char ch, Node *node)
```

```
    {  
        links[ch - 'a'] = node;
```

```
    }  
    Node *get(char ch)
```

```
    {  
        return links[ch - 'a'];
```

```
    }  
};
```

```
int
```

```
countDistinctSubstrings(string s)
```

```
{  
    int cnt = 0;
```

```
    Node *root = new Node();
```

```
    for (int i = 0; i < s.size(); i++)
```

```
    {  
        Node *node = root;
```

```
        for (int j = i; j < s.size(); j++)
```

```
        {  
            if (!node->containsKey(s[j]))
```

```
            {  
                cnt++;
```

```
                node->put(s[j], new Node());
```

```
                node = node->get(s[j]);
```

```
            }  
        }  
    }
```




Date / /
Page

return cont+1;

Bit Prerequisite For Tries

9 → 1001 X → computer doesn't store this way

0000 - - - - 1001
32 bit 164 bits

✓. ✓. ✓. ✓

Q. Power set

can also be solved using recursion

Find all possible subsequences of string in lexicographically sorted order

ex:- str = "abc"

Q5) o/p:- a ab abc ac b bc c → 7

str = "aa"

o/p:- a a aa → 5

Q6) s = "abc" → "", a, b, c, ab, ac, bc, abc

n = 3

$2^3 = 8$

5 → 1 0 1

rd (C/D)

rd (C/D)



Date _____
Page _____

0 1 2

0 → 000 → "" s = "abc" → "a, b, c, ab, ac, bc, abc"
1 → 001 → a
2 → 010 → b
3 → 011 → ab
4 → 100 → c
5 → 101 → ac
6 → 110 → bc
7 → 111 → abc

$O(2^n)$

For (num = (0 → $2^n - 1$))

sub = ""

For (i = 0 to n-1)

$O(n)$

if (num & (1 << i))

sub += s[i]

Print (sub);

TC → $O(2^n \times n)$

SC → $O(1)$

Base 2

QT
Q

Given an array of numbers, & a number x.
Find the maximum value of (arr[i] ^ x)

→ Insert all the numbers into the trie
(binary bit)

→ Take x & find the max no. for array where (no ^ x) ↑ ↑ max

9 → (000 - - 1001)

3 for ex p¹

9 → 01001

5 bits

Nodes

links [2];

b, ac, bcd

$(1 \ll i) - 1$

1)

1)

$(1 \ll i)$

$[i]$

Q_i

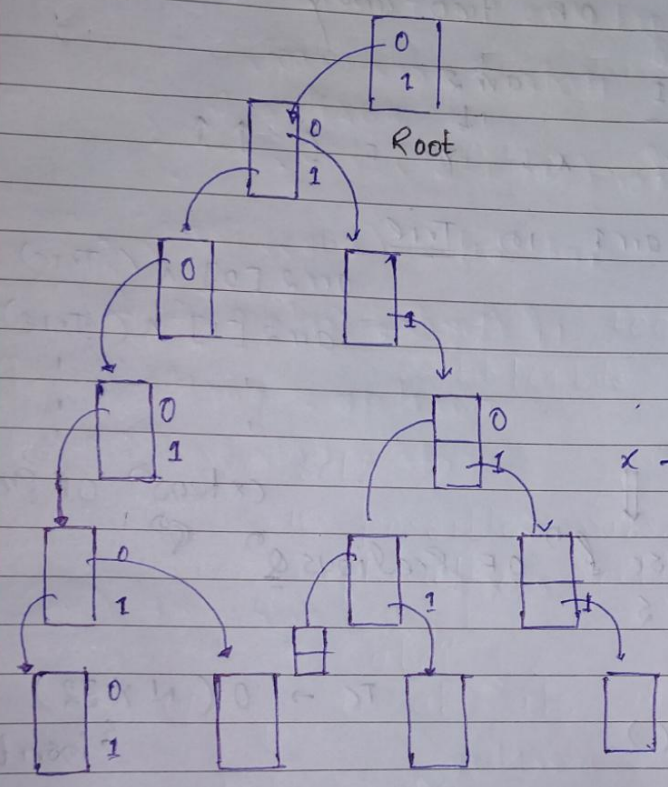
number
arr[i] ^ x

into the
binary bits

no. for
two max

bits

9, 8, 7, 5, 4



9 → 01001
8 → 01000
7 → 00111
5 → 00101
4 → 00100

x = 8

x → 01000

00111

01111

intuition

maximizing XOR

x = 8 ^ arr[i] =

8 = 01000

you want 10111

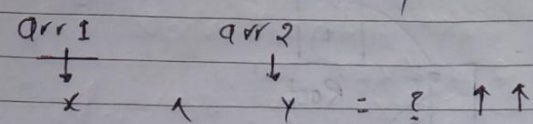
11111

entire input

This should be largest



Q. Given two array find maxⁿ XOR value of any pair of two array



Put arr1 in Trie

arr2[0] ^ (Trie)
arr2[1] ^ (Trie)
⋮
⋮

extension of previous

use fⁿ of previous Qⁿ

class Trie {

root

insert()

int getMax()

}

code:-

struct Node {

Node * links[2];

bool containKey(int bit) {

return (links[bit] != NULL);

}

TC → O(N × 32)

{ insertⁿ

+ O(M × 32)

{ for find maxⁿ



```
Node* get (int bit) {  
    return link[bit];  
}  
  
void put (int bit, Node* node)  
{  
    link[bit] = node;  
}
```

6 i

class Tries

private:

Node* root;

Tries() { root = new Node(); }

public:

void insert (int num) {

for (int i = 31; i >= 0; i--) {

int bit = (num >> i) & 1; → ith bit
is set or
not

if (! node->containsKey(bit)) {

node->put (bit, new Node());

node = node->get (bit);

}

```

int getMax (int num) {
    Node * node = root;
    int maxNum = 0;
    for (int i = 31; i >= 0; i--) {
        int bit = (num >> i) & 1; // opposite
        if (!node->contain(1-bit)) {
            maxNum = maxNum | (1 << i);
            node = node->get(1-bit);
        }
        else {
            node = node->get(bit);
        }
    }
    return maxNum;
}

```

```

int maxXOR (int n, int m, vector<int> &arr1,
            vector<int> &arr2) {
    Trie trie;
    for (auto &it : arr2) trie.insert(it);
    int maxi = 0;
    for (auto &it : arr1) {
        maxi = max (maxi, trie.getMax(it));
    }
    return maxi;
}

```